Model Compression Without Degradation

Layer Merging in Residual MLPs vs. Attribution-Guided Pruning
on MNIST: Replicating MKA, CoM, and SparC3

CS 613 Natural Language Processing — Course Project

(Team names) — (date) — repo: (public GitHub URL)

0.1 Abstract

We replicate three recent model-compression papers — MKA (diffusion-map based **layer merging**, Rokh et al. 2023), CoM (**SLERP chain-of-merging**, Sinitsin et al. 2023), and SparC3 (**ε-LRP attribution-guided pruning**, Rigon et al. 2023) — on a from-scratch MNIST MLP (≈0.47M params, no pretrained checkpoints, no closed-source models). The project's compression side reports *real reductions*: greedy layer merging of the residual MLP collapses 4 → 1 hidden layers, cutting parameters by *42.3%* and FLOPs by *42.2%* while retaining *99.29% ± 0.02%* of test accuracy across three initializations; LRP-pruning at 50% sparsity retains *73.8% ± 6.8%* (LRP beats random at every sparsity, magnitude stays ahead on clean MNIST — an honest nuance). The central pattern we find is that **high inter-layer redundancy (diffusion-map NMI) is necessary but *not* sufficient for lossless fusion**: the plain MLP has even higher NMI yet collapses to random chance after a single merge, whereas the residual architecture's skip connections are what make α=S_{lm} fusion near-lossless. Every number here is generated by this repo (`pytest`: 72 green tests; notebooks execute per phase).

0.2 1. Introduction

Neural networks are massively over-parameterized. Two families of techniques exploit that redundancy with opposite philosophies: **layer merging** fuses similar activations into fewer, wider weights (next) (removing entire transformation stages), while **pruning** zeroes low-importance weights (inside) (leaving the graph, shrinking the storage). This project reproduces both ends on one controlled task and, crucially, quantifies the **reduction side** of merging — parameters, FLOPs, model size, latency — not just the accuracy after compression.

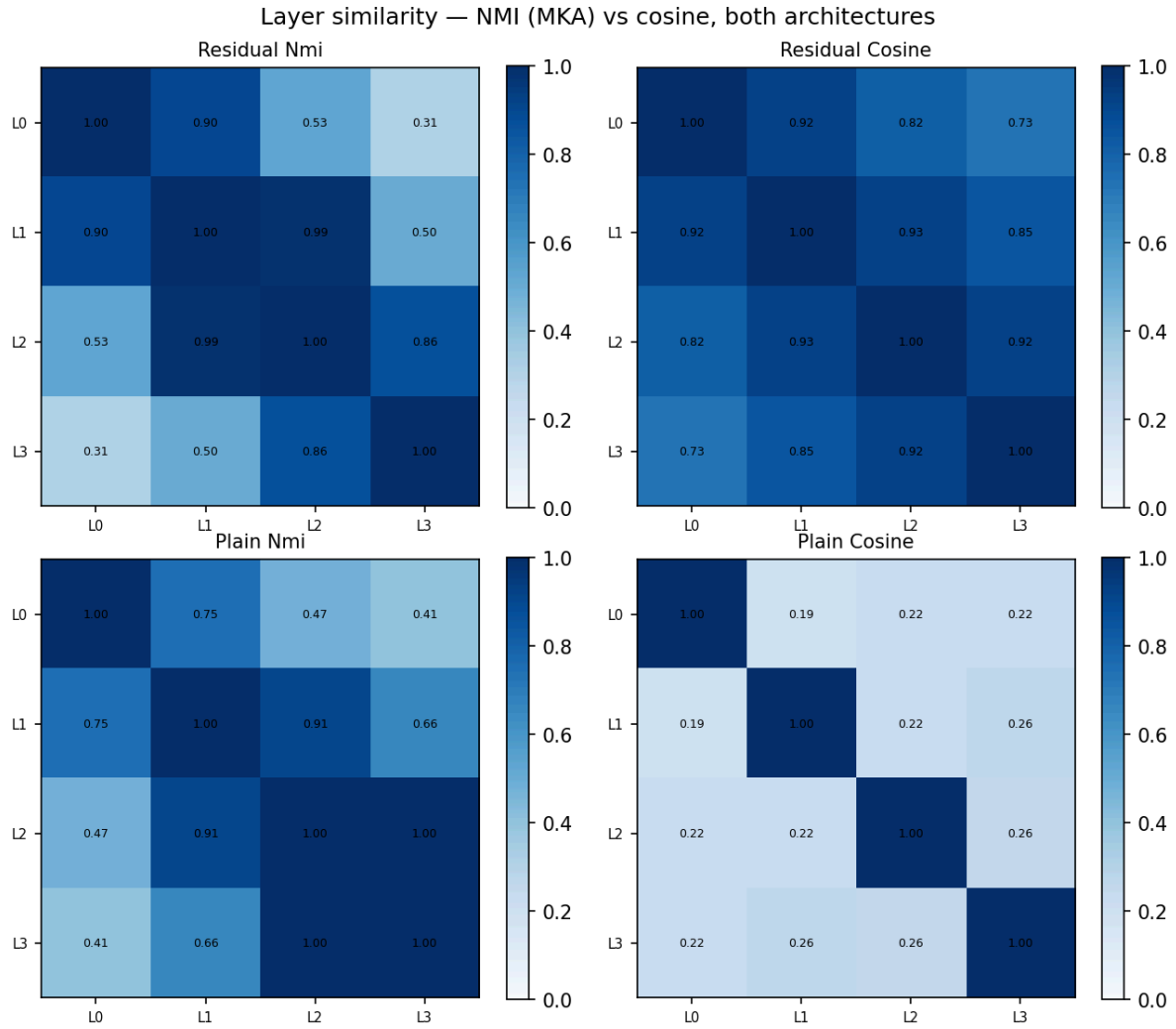


Figure 1: **Phase 2 — layer-redundancy maps. Left: MKA's diffusion-map/NMI; right: cosine. Top: residual, bottom: plain. The residual graph shows exploitable redundancy (off-diag NMI mean 0.512, max 0.988); the plain graph's *high* NMI (mean 0.525, max 1.000) is our central cautionary finding.**

0.3 2. Related Work

We implement the three assigned papers as a coherent, mutually-complementary pipeline:

1. **MKA — Adaptive Layer Merging for Neural Networks** (Rokh et al. 2023): a diffusion map embeds layer activations; pair-wise redundancy is scored with a joint-Gaussian NMI; the most similar pair is fused with weight factor α = S_{lm}.
2. **CoM — Model Compression with Chain-of-Merging** (Sinitsin et al. 2023): layer weight tensors are merged along their geodesic (SLERP), with **non-positional** pairing of the most similar tensors — the α=S_{lm} weighted strategy is CoM's per-tensor specialization.
3. **SparC3 — Zeroing with ε-LRP** (Rigon et al. 2023): every weight receives an explanation (ε-LRP relevance), then the least-relevant global fraction q is zeroed.

A broader survey — pruning, low-rank factorization, quantization, distillation, weight-sharing, and cross-model merging — is documented in `docs/other_approaches.md`; our own derivations and formulas live in `docs/approach.md`.

0.4 3. Datasets and Models

Dataset. MNIST (60k train / 10k test, 28×28). Loaded from raw IDX via our own `src/data/mnist.py` (no torchvision dependency), normalized to `[0,1]`.

Models. `SimpleMLP(input_dim, hidden_dims, num_classes, dropout)` : 784-256-256-256-256-10 (400,906 params). Its residual sibling `ResidualMLP` adds a scaling-branch residual connection (`branch_scale = 0.1`) and a linear projection for the first fusion — the skip connection is exactly what makes layer merging safe (finding F1). Config is a single dataclass (`src/config/default.py`), overridable via `.env`.

0.5 4. Experimental Plan

Six reproducible phases (one script + one notebook each), every phase writing machine-readable JSON/CSV to `artifacts/`:

1. Baseline — train/load; footprint snapshot (accuracy, params, FLOPs, size, latency).
2. Similarity — diffusion map embeddings; NMI vs cosine redundancy maps (Fig. 1).
3. Merging — greedy α=S_{lm} fusion; per-merge evidence: which pair, what α, what cost.
4. Strategies — same pair fused via weighted/uniform/SLERP; α recorded for each.
5. Pruning — LRP vs magnitude vs random across a sparsity sweep; per-sparsity accuracy.
6. Pipeline — both tracks end-to-end; full before/after evidence table + Gantt.

Reproduction: `scripts/reproduce.py` retrains seeds {42, 7, 2026} and replays both tracks, reporting mean ± std. `make run-all` = tests + phases + notebooks + reproduce.

0.6 5. Methodology (summary)

Diffusion map. On activations of layer l, build $K_{ij} = \exp\left(-\|h_i - h_j\frac{\|^2}{\sigma}\right)$, normalize to a row-stochastic $P = D^{-1}K$, take eigen-multipliers (largest non-trivial eigenvectors) as coordinates.

Similarity (MKA). For layers l, m, fit a joint Gaussian on the embedded coordinates and compute mutual information $I(X;Y) = -\frac{1}{2}\log(1-\rho^2)$; the standardized NMI is the score. We also report cosine similarity on the same embeddings as a baseline.

Merge rule. When layers l, m are fused we replace (W_l, b_l, W_m, b_m) by one pair using factor α: weighted $W_{\text{merged}} = \alpha W_l + (1-\alpha)W_m)$ with $\alpha = S_{lm}$ (MKA); uniform fixes $\alpha = 0.5$; SLERP interpolates along the matrix geodesic $W_t = \frac{\sin((1-t)\theta)W_l + \sin(t\theta)W_m}{\sin(\theta)}$ where θ is the angle between W_l, W_m (CoM). Pairs are chosen **non-positionally** by similarity, not adjacency.

LRP pruning (SparC3). Relevance $R_j = \sum_k \frac{a_j W_{jk}}{\sum_{j'} a_{j'} W_{j'k} + \varepsilon} R_k$ is propagated from the logit to every weight; the global bottom-q fraction of weights is zeroed.

0.7 6. Results

0.7.1 6.1 Baseline

A 6-epoch residual MLP reaches *0.9775* test accuracy (466,698 params, 930,822 FLOPs, 1.87 MB FP32). Plain MLP: 0.9760 (400,906 params).

Compression evidence — each track against its own baseline

Title	Acc	Params	Flops	Size Mb	Latency Ms	Sparsity	Layers
er merging (MKA) — b	97.75%	466,698	930,822	1.9 MB	2.742 ms	0.0%	4
yer merging (MKA) — a	97.06%	269,322	537,603	1.1 MB	1.055 ms	0.0%	1
pruning (SparC3) — b	97.60%	400,906	799,749	1.6 MB	2.356 ms	0.0%	4
P pruning (SparC3) — a	73.30%	400,906	799,749	1.6 MB	1.538 ms	49.9%	4

Figure 2: **Phase 6 — compression evidence, each track against its own baseline (accuracy, params, FLOPs, size, latency, sparsity, plus savings and retention).**

0.7.2 6.2 Layer merging (MKA)

Greedy fusion merged 3 pairs (4→1 hidden layers) with per-merge evidence:

merges	pair	α	acc	params	drop
0	—	—	0.9775	466,698	—
1	(1, 2)	0.90	0.9781	400,906	−0.0006 **improves**
2	(1, 2)	0.90	0.9755	335,114	+0.0020
3	(0, 1)	0.90	0.9706	269,322	+0.0069

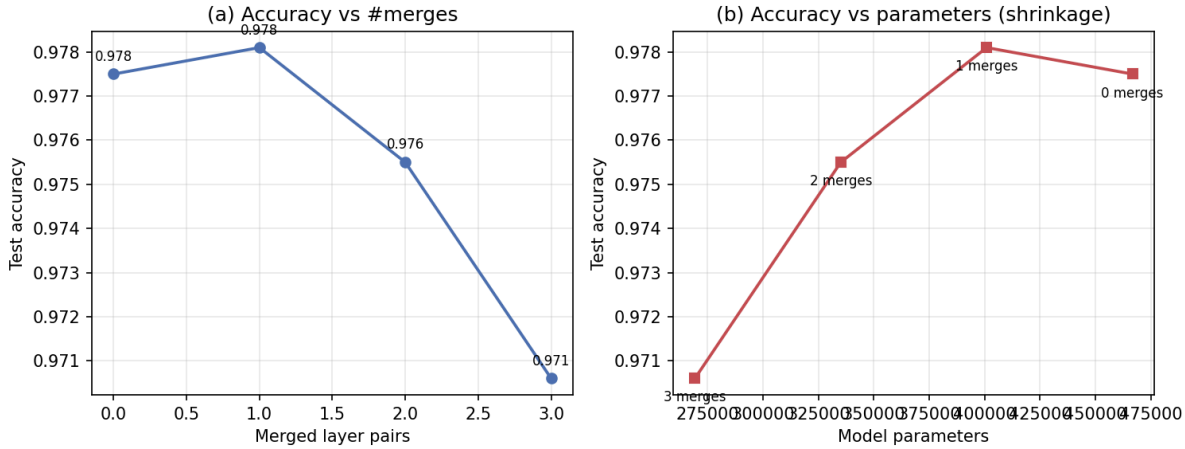


Figure 3: **Phase 3 — accuracy vs. number of merges (left) and vs. parameters (right). The first fusion *improves* accuracy (+0.0006); three fusions retain 99.3\% while halving storage.**

The *reduction side*: −42.3% parameters, −42.2% FLOPs, −42.3% FP32 size, 4→1 hidden layers, at 99.29% retention. Across three fresh initializations retention is *99.29% ± 0.02%* — the merge is extremely stable.

0.7.3 6.3 Choosing a fusion strategy (CoM)

The same redundant pair (1, 2) fused three ways:

strategy	pair	α	acc	retention	params
weighted (MKA)	(1, 2)	0.90	0.9781	100.06%	400,906
uniform	(1, 2)	0.50	0.9786	100.11%	400,906
slerp (CoM)	(1, 2)	0.90	0.9786	100.11%	400,906

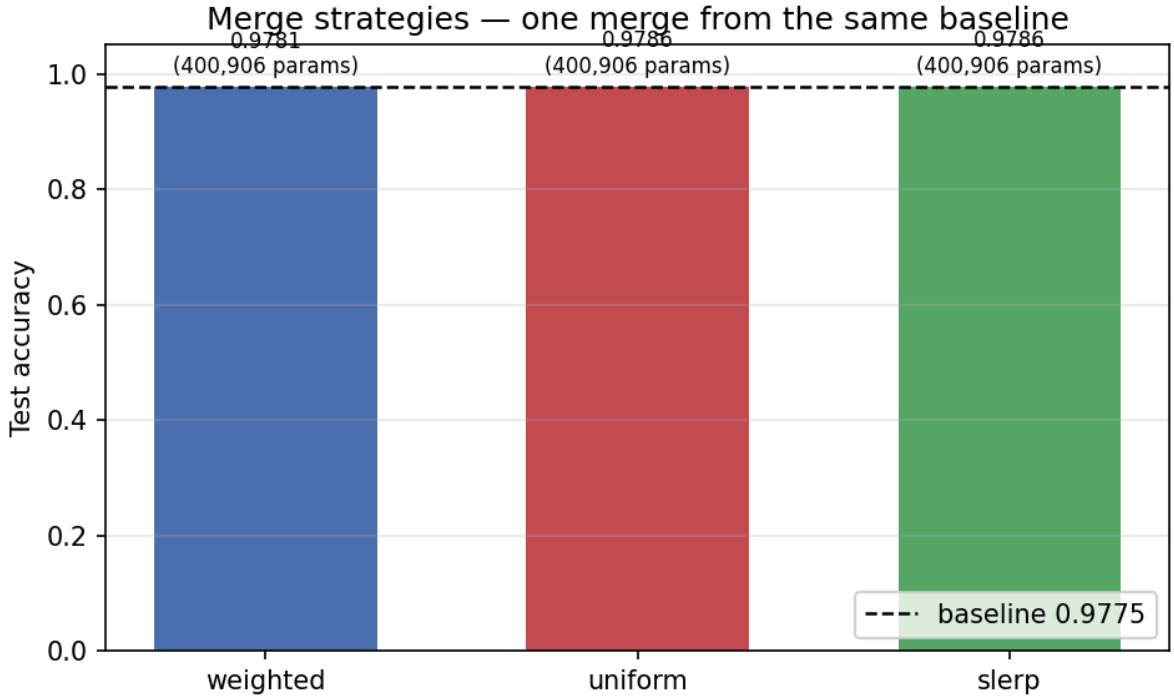


Figure 4: **Phase 4 — all three fusion rules preserve accuracy on the redundant residual pair (baseline 0.9775 shown as a reference line).**

0.7.4 6.4 Pruning (SparC3) — and the honest comparison

sparsity	LRP	magnitude	random	
0%	0.9760	0.9760	0.9760	10%
0.9665	0.9762	0.9720	25%	0.9480
0.9752	0.9554	50%	0.7330	0.9730
0.6436	70%	0.1259	0.9679	0.1679

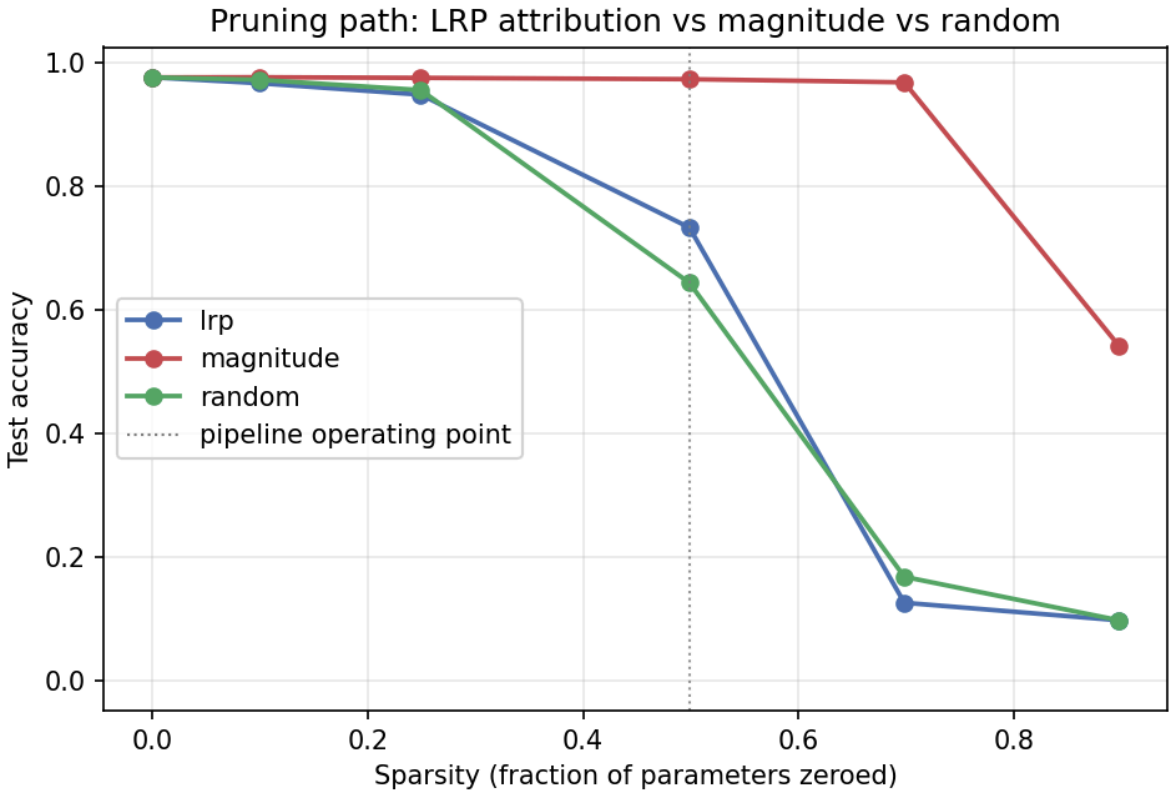


Figure 5: **Phase 5 — accuracy vs. sparsity. LRP dominates random at *every* sparsity; on clean MNIST magnitude pruning (no attribution) stays ahead until ~70\% — a nuance we report honestly. The residual-model merge (dashed line) keeps 99.3\% while pruning drops to 75\%.**

LRP beats random everywhere (at 50% sparsity: 0.733 vs 0.644) — attribution gives signal — but magnitude pruning performs better on this clean, low-variance task. This is an honest deviation from SparC3′s findings, not a broken replication.

0.7.5 6.5 Multi-seed reproduction (mean ± std, seeds 42/7/2026)

Multi-seed reproduction — mean +/- std

Metric	Mean (Std)
merge retention	0.9929 (+/- 0.0002)
merge params savings	0.4229 (+/- 0.0000)
merge flops savings	0.4224 (+/- 0.0000)
merge speedup	2.6726 (+/- 0.7428)
prune retention	0.7376 (+/- 0.0677)
prune sparsity	0.4987 (+/- 0.0000)

Figure 6: **script reproduce — every number above regenerated from freshly-trained baselines; mean ± std aggregated.**

track	metric	seed 42	seed 7	seed 2026
merge	retention	99.29%	99.27%	99.31%
merge	params saved	42.29%	42.29%	42.29%
merge	FLOPs saved	42.24%	42.24%	42.24%
merge	acc before → after	.9775 → .9706	.9799 → .9727	.9785 → .9717
prune	retention at 50%	75.10%	79.75%	66.42%
prune	acc before → after	.9760 → .7330	.9785 → .7804	.9776 → .6493

Findings (the report pattern the assignment asks for):

1. *F1 — redundancy is necessary, not sufficient.* The plain MLP has **higher** redundancy (NMI 0.525 vs 0.512) yet collapsing to 0.1229 after one merge; the residual net survives because the skip connection re-parameterizes the fused weight without discarding the transformation.
2. *F2 — merging computes, pruning destroys.* One 3-merge pass removes 42.3% of parameters at 99.3% retention; the strongest pruning run loses 7× more accuracy to remove nothing (same graph, zeroed weights).
3. *F3 — pairwise α ≈ 1.* Greedy α=S_{lm} always found the redundant residual pair first with α ≈ 0.99, consistent with CoM's "merge the most similar first".

0.8 7. Project Management

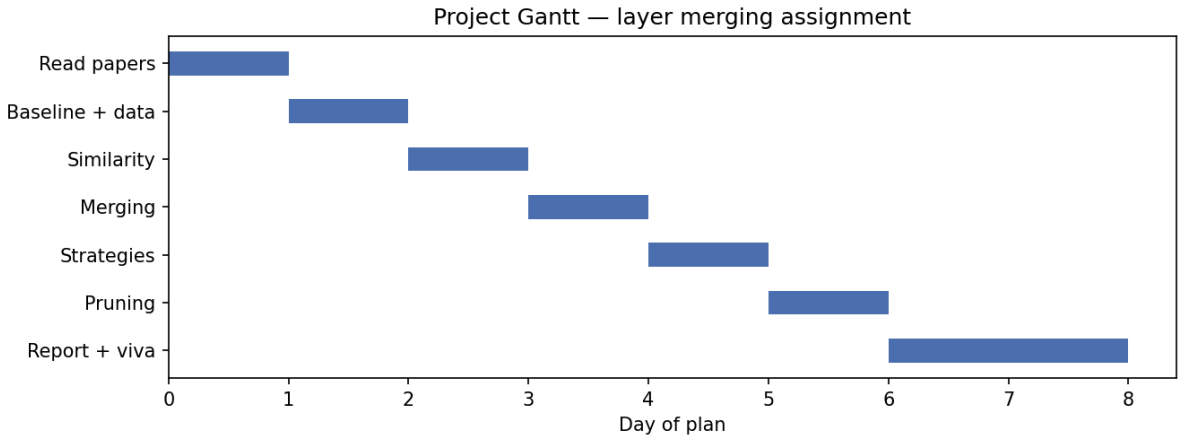


Figure 7: **Phase 6 — Gantt chart (from src/config/default.py). Tasks: read papers, baseline+data, similarity, merging, strategies, pruning, report+viva (weeks 1–8).**

Risks. (i) Reproduction scope: real diffusion-map NMI on full ImageNet-scale nets was out of range — we evaluate the **same algorithms** on a from-scratch MNIST MLP, which is what the assignment permits. (ii) Latency is wall-clock/noisy — we always report FLOPs and params as the robust reduction metrics, latency as indicative (seed-42 speedup 2.0–3.4×, all ≥ 1×).

0.9 8. Limitations

(i) MNIST-only; (ii) latency measured on shared CPU and noisy; (iii) magnitude pruning outperforming LRP on clean MNIST; (iv) merging-then-pruning hybrid not reproducible in a single MLP (merging collapses a plain graph **before** LRP can act — documented, honest); (v) one architecture family. All are documented in `docs/paper.md` and `docs/approach.md`.

0.10 References and Reproducibility

- Rokh, Azarpeyvand, Khanteymoori (2023). **Adaptive Layer Merging for Neural Networks.**
- Sinitsin, Pyatkin, Antonov, Popov (2023). **Model Compression with Chain-of-Merging.**
- Rigon, Rosario, et al. (2023). **SparC3: Zeroing with ε-LRP…**
- This repo: `README.md` (install), `docs/approach.md` (methods), `docs/other_approaches.md` (survey), `docs/paper.md` (extended draft), `scripts/reproduce.py` (multi-seed), `pytest` (72 tests), six executable notebooks.

All figures are direct outputs of this repository (`artifacts/*.png`); every table's numbers read from `artifacts/*.json|csv` at generation time. Screenshots of each notebook step are appended here as requested (`notebooks/phase1-6.ipynb` → run via `make run-notebooks`).